Results

We compared four deep-learning architectures on multiclass human activity recognition: a fully connected network (DNN), a plain recurrent network (RNN), a 1D convolutional network (CNN) and an LSTM. All four get the same input, the 561 engineered features of each sample, and have to predict one of six activities (LAYING, SITTING, STANDING, WALKING, WALKING_DOWNSTAIRS, WALKING_UPSTAIRS).

Experimental setup and hyperparameter choices

Data split and preprocessing. We split the original training set into training (5551 samples, 16 subjects) and validation (1801 samples, 5 subjects) using `GroupShuffleSplit` by subject. Since nobody appears in both sets, the validation score tells us how well a model works on people it has never seen, which is what matters in practice. The features are standardised with a `StandardScaler` fitted on the training split only. We kept the test set (2947 samples) aside for the final evaluation and did not use it to pick models.

Training configuration. To keep the comparison fair, every model was trained with the same settings:

- **Optimiser: Adam, learning rate 10^{-3}.** Adam adjusts the step size for each parameter, and the default learning rate usually works for dense, convolutional and recurrent models alike. For a first comparison we did not tune it per model.
- **Batch size 64.** That is roughly 87 updates per epoch with 5551 samples. It is enough to converge in a few dozen epochs, the gradient noise helps a bit as regularisation, and the GPU is still used well.
- **Up to 50 epochs, early stopping (patience 8), restoring the best weights.** 50 is just a ceiling; no model got there. Early stopping watches the validation loss. We used a patience of 8 because the CNN and the recurrent models have quite noisy validation curves and a smaller value stopped them too soon. The weights from the best validation epoch are restored at the end, so the choice of model depends only on validation data.
- **Gradient clipping (norm 1.0).** The RNN and LSTM are unrolled over 561 steps and can easily suffer from exploding gradients. Clipping prevents this and does not hurt the other models.
- **Dropout 0.3 in all models,** a middle value that regularises without slowing training down much.
- **Seed 42** in the main comparison (initial weights, shuffling and dropout).

Architectures. We kept the models small so each one trains in seconds, but with enough capacity for the task:

- **DNN:** 561 → 256 → 128 → 6, with BatchNorm, ReLU and dropout after each hidden layer. The layers get narrower step by step, and BatchNorm makes training more stable and faster.
- **RNN and LSTM:** a single layer with hidden size 64. Each sample is fed as a sequence of 561 steps (one value per step) and a linear layer classifies the last hidden state. We used one layer because every extra layer means another pass over the 561 steps.
- **CNN:** three Conv1d → BatchNorm → ReLU → MaxPool(2) blocks with 32, 64 and 128 filters (kernel size 5), then global average pooling. Every pooling halves the length (561 → 280 → 140 → 70), so we double the filters each time. Global average pooling replaces what would otherwise be a big dense layer and keeps the model small.

Everything was trained on an NVIDIA RTX 3050 Laptop GPU. All metrics reported below are on the test set.

Main comparison

Model	Accuracy	Macro Precision	Macro Recall	Macro F1	Weighted F1	Parameters	Training Time	Inference Time
DNN	0.9525	0.9542	0.9510	0.9520	0.9524	178310	6.63 s	0.0029 s
RNN	0.7642	0.7594	0.7542	0.7557	0.7650	4678	9.66 s	0.0197 s
CNN	0.8487	0.8632	0.8487	0.8453	0.8486	52806	9.14 s	0.0456 s
LSTM	0.7913	0.7943	0.7850	0.7868	0.7919	17542	11.20 s	0.0466 s

Table 1: Test metrics, trainable parameters, total training time and inference time on the test set.

Predictive performance

The **DNN** wins by a wide margin: 0.952 accuracy and 0.952 macro F1. Next is the **CNN** (macro F1 0.845), then the **LSTM** (0.787) and last the **RNN** (0.756). Even the second-best model is about 10 macro F1 points behind the DNN.

Macro and weighted F1 are almost the same for all models, and so are precision and recall. This makes sense because the test set is fairly balanced (420 to 537 samples per class), so no class dominates the averages.

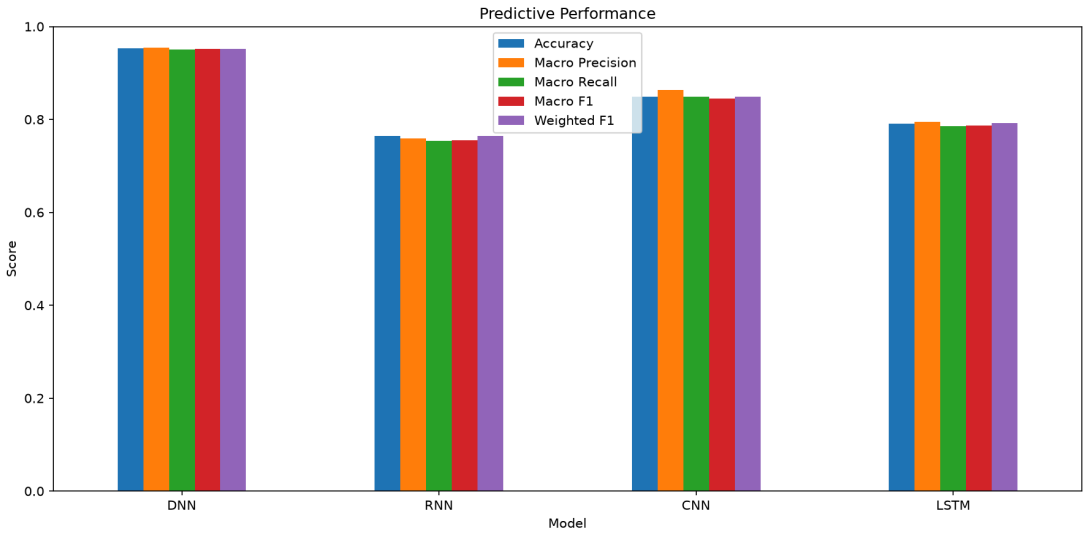


Figure 1: Predictive metrics per model.

Our interpretation is that, with these features, it is better to treat the 561 values as one vector than as a sequence. They are hand-crafted statistics (means, energies, frequency coefficients…), and the order in which they appear has no temporal meaning, which is precisely what recurrent and convolutional layers rely on.

Computational cost

The DNN has many more parameters than the rest (178,310, compared with 52,806 for the CNN, 17,542 for the LSTM and 4,678 for the RNN). Even so, it is the fastest one, both in training (6.6 s) and at inference (0.003 s for the whole test set). The reason is that a dense layer handles the whole input with one matrix product, which GPUs do very quickly. At inference, the CNN and the LSTM take roughly 15 times longer than the DNN.

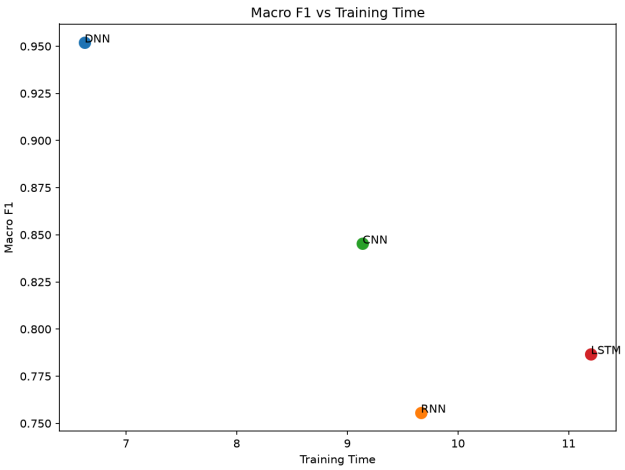


Figure 2: Macro F1 vs. training time.

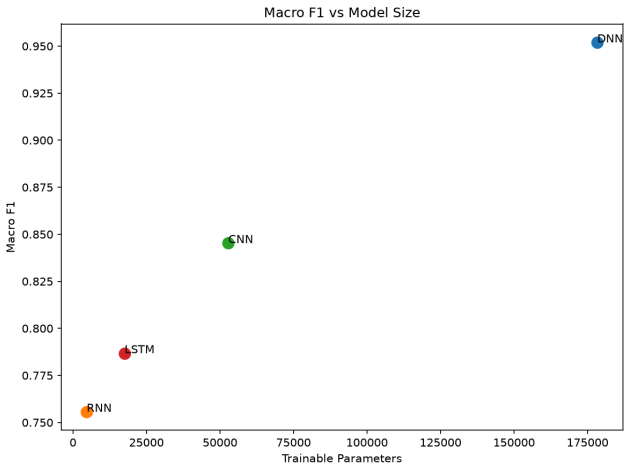


Figure 3: Macro F1 vs. number of trainable parameters.

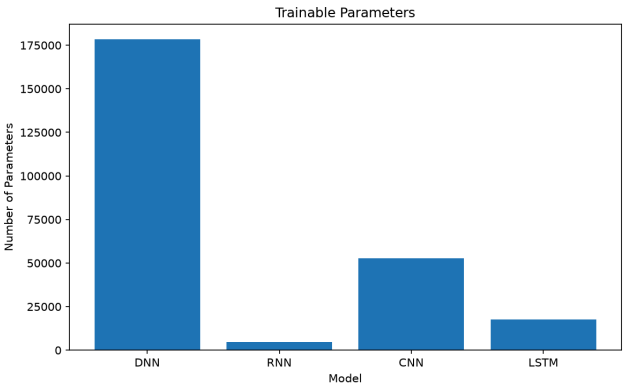


Figure 4: Trainable parameters.

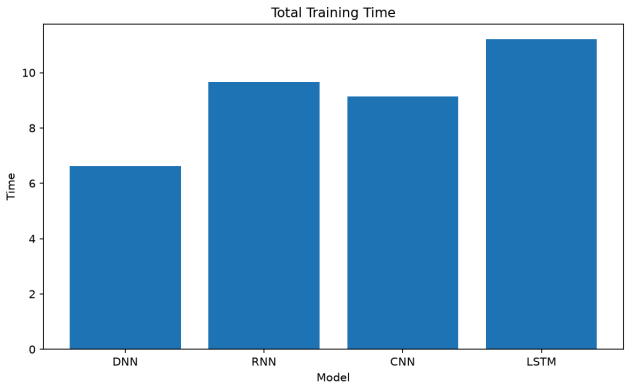


Figure 5: Total training time.

How long training takes depends on the time per epoch and on when early stopping kicks in. The RNN has the cheapest epochs (0.28 s) but ran for 34 of them, whereas the DNN and the CNN stopped at 20 and the LSTM at 27. None of them hit the 50-epoch limit.

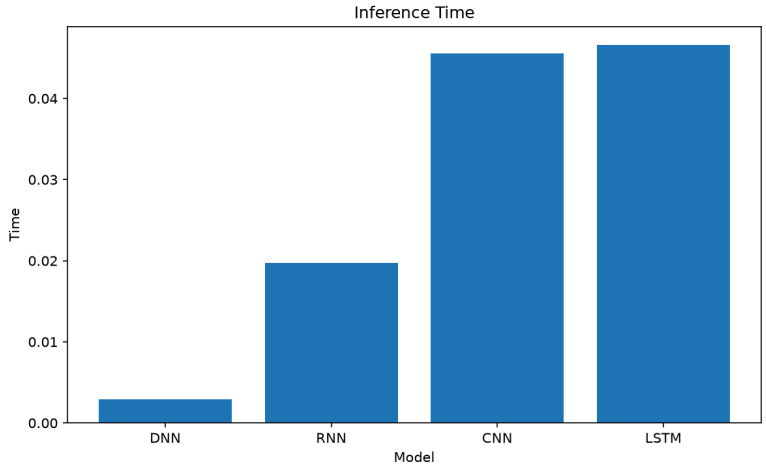


Figure 6: Inference time on the test set.

So here, more parameters does not mean more cost. The DNN is both the most accurate and the cheapest model, and the recurrent models, though much smaller, are slower and worse.

Training curves

The DNN converges within a few epochs and its validation loss stays low, close to the training loss. The RNN and the LSTM also improve steadily, but more slowly, and their validation loss jumps around more.

The CNN is the least stable. Its training loss keeps going down while the validation loss swings a lot, reaching peaks of about 1.7. We think this is due to the learning rate being too high for it, or to some overfitting. It would also explain why its test results are lower than its training accuracy suggests.

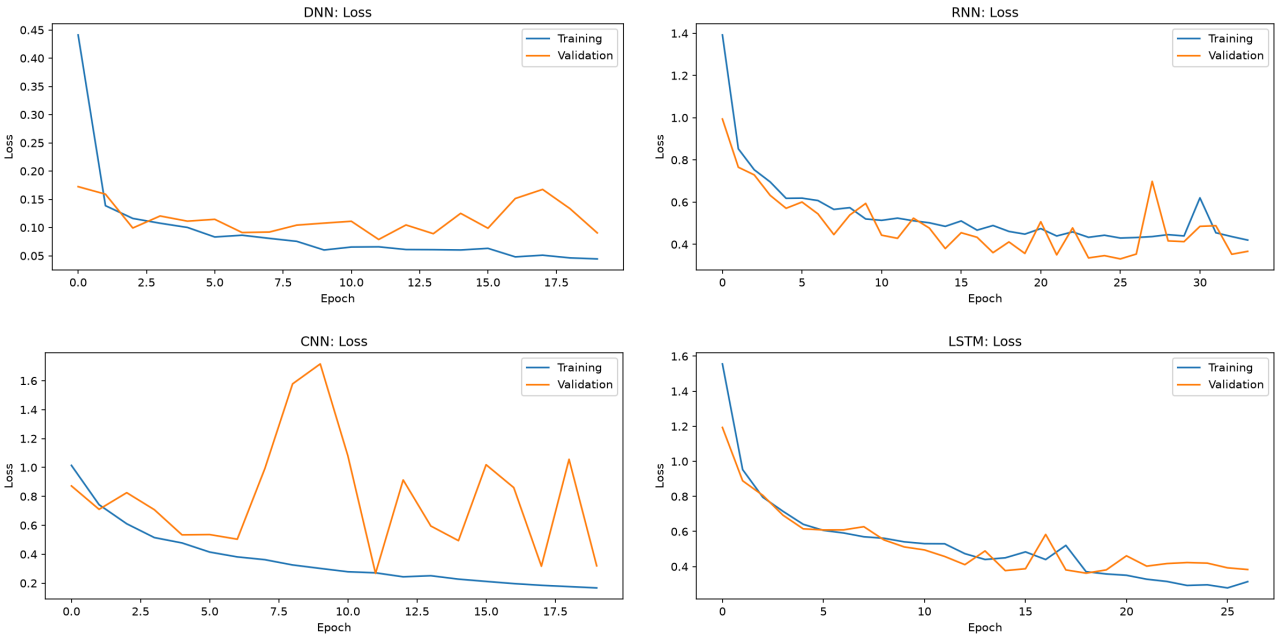


Figure 7: Training and validation loss (DNN, RNN, CNN, LSTM).

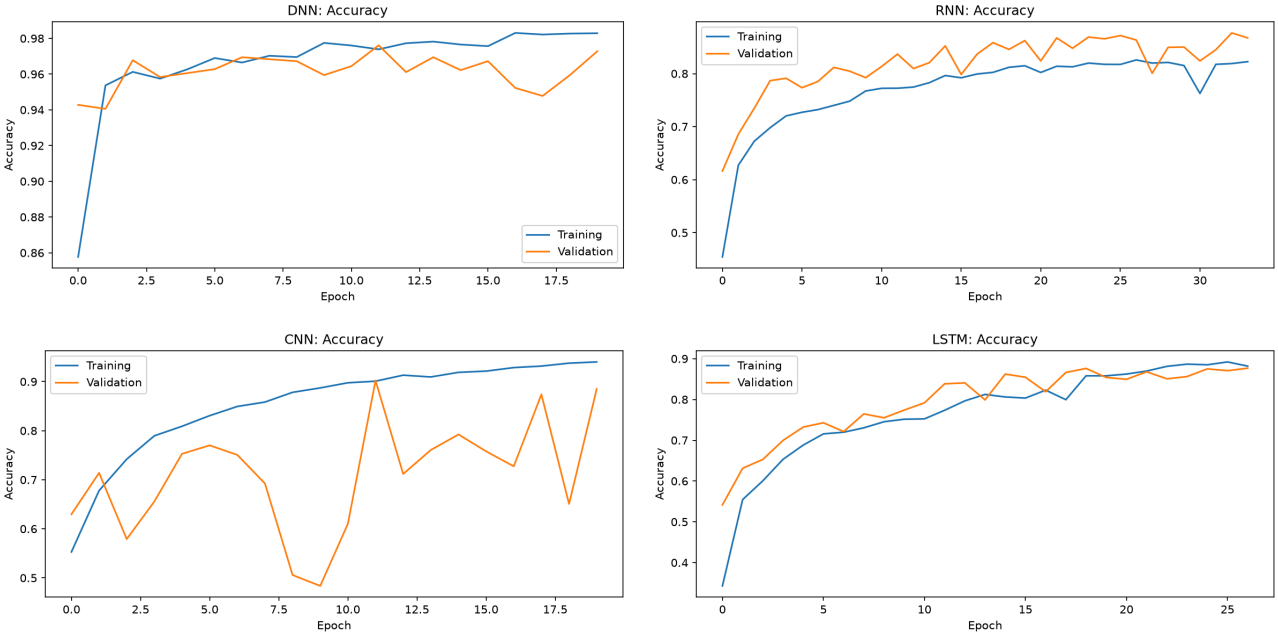


Figure 8: Training and validation accuracy (DNN, RNN, CNN, LSTM).

Confusion matrices

All models get LAYING almost perfectly right (the DNN gets all 537 samples). Most mistakes happen between activities that look alike:

- **Static (SITTING vs. STANDING).** Even the DNN predicts STANDING for 56 SITTING samples. The LSTM makes the same mistake 114 times, while the CNN does the opposite and labels 133 STANDING samples as SITTING.
- **Dynamic (WALKING, WALKING_UPSTAIRS, WALKING_DOWNSTAIRS).** Here is where the recurrent models lose the most. The RNN only gets an F1 of 0.57 on WALKING_DOWNSTAIRS, and the LSTM predicts WALKING for 139 WALKING_UPSTAIRS samples. The CNN has trouble with WALKING_UPSTAIRS too (recall 0.66).

Static and dynamic activities are rarely confused with each other (the DNN almost never does it). So every model learns this basic split, and what separates them is how well they tell apart activities inside each group.

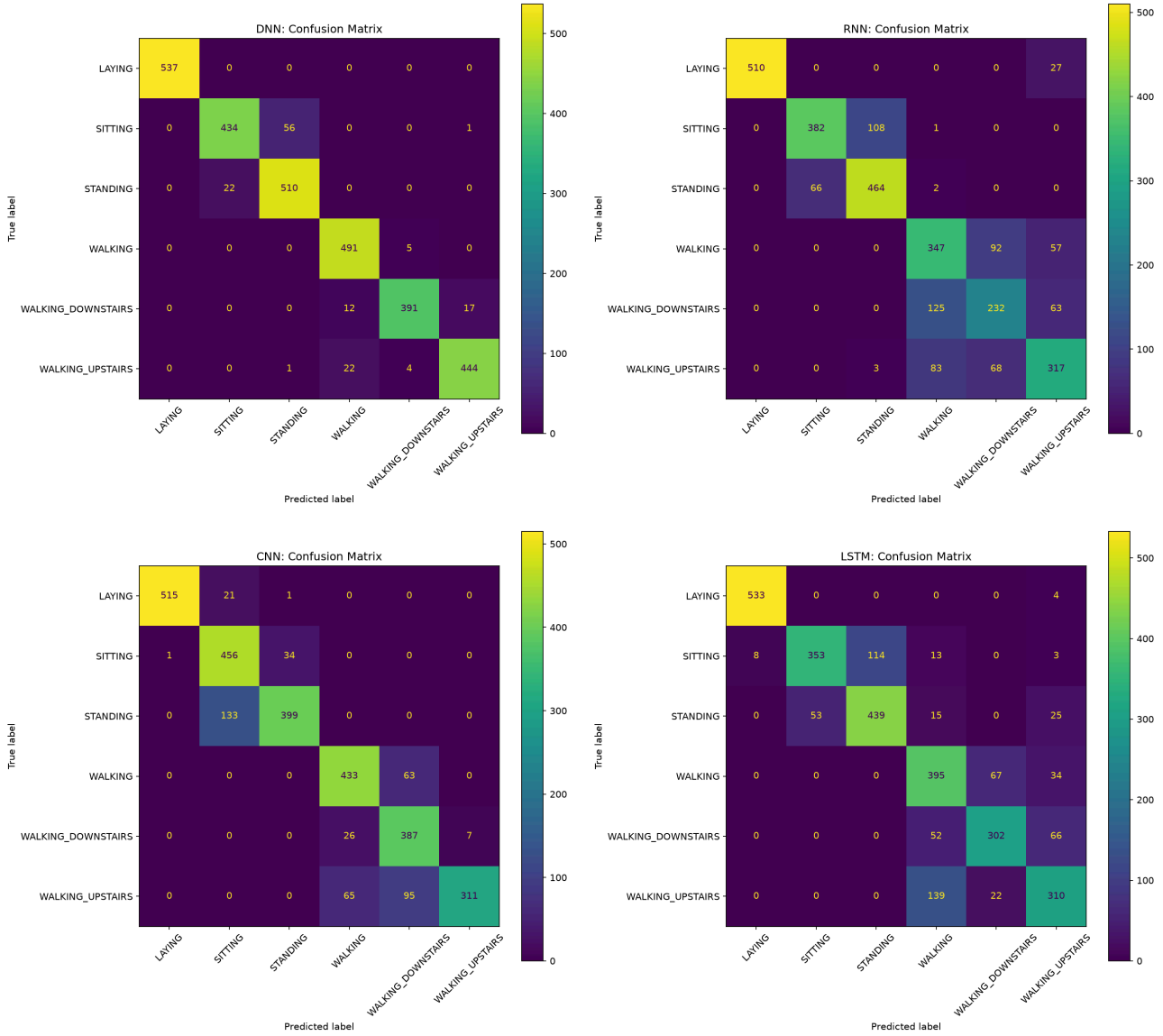


Figure 9: Test confusion matrices (DNN, RNN, CNN, LSTM).

Repeated runs with three seeds

Motivation. The result of one training run depends on chance: initial weights, mini-batch order and dropout masks. We wanted to know if the ranking above comes from the architectures or just from seed 42, so we trained the four models again from scratch with seeds 1, 42 and 123 (new models each time).

What we expected. Models that are easy to optimise and fit the data well should give similar numbers with any seed. The ones trained on long sequences (RNN, LSTM) or with an unstable validation loss (CNN) should vary more, since small differences at the start of training can end in different solutions and in early stopping at a different epoch.

Model	Accuracy	Macro F1	Training Time (s)	Epochs
DNN	0.9421 ± 0.0093	0.9420 ± 0.0090	5.12 ± 0.93	13, 20, 15
RNN	0.7647 ± 0.0209	0.7559 ± 0.0205	9.20 ± 0.72	28, 34, 31
CNN	0.8298 ± 0.0205	0.8194 ± 0.0254	10.34 ± 0.87	22, 20, 23
LSTM	0.8198 ± 0.0287	0.8146 ± 0.0282	12.89 ± 0.63	31, 27, 26

Table 2: Mean ± standard deviation over seeds 1, 42 and 123 (test set). Epochs are listed per seed.

Results. The DNN comes first with all three seeds (macro F1 from 0.935 to 0.952) and has the smallest standard deviation (0.009), so its lead is much bigger than the noise between runs. The RNN is always last. The CNN and the LSTM do swap places, though. With seed 42 the CNN is clearly ahead (0.845 vs. 0.787), but with seeds 1 and 123 the LSTM is better, and on average they are practically tied (0.819 vs. 0.815). Their standard deviations (0.025–0.028) are about as large as the gap between them.

The number of epochs also changes with the seed (13 to 20 for the DNN, 28 to 34 for the RNN), and training times change with it. With seed 42 we get exactly the same numbers as in the main comparison, so the pipeline is reproducible.

Conclusion. The big differences (DNN best, RNN worst) hold up, but one run is not enough to compare models that are only a few points apart, like the CNN and the LSTM. Had we only used seed 42, we would have said the CNN is clearly better than the LSTM, and the other two seeds show that is not the case.

Controlled experiment: LSTM hidden size

Parameter and motivation. We tried LSTM hidden sizes of 32, 64, 128 and 256 and left everything else unchanged (data, optimiser, learning rate, batch size, early stopping, seed 42). We picked this parameter because it controls how much the LSTM can remember: the whole 561-step sequence has to fit into one hidden state of that size. It also matters a lot for cost, as the parameter count grows roughly with the square of the hidden size ($4h(h+3)+6h+6$ with one input value per step).

What we expected. A bigger hidden state can keep more information about the sequence, which might close part of the gap with the DNN. But a bigger model can also overfit the 16 training subjects, and each recurrent step gets more expensive, so training and inference times should go up fast.

Hidden Size	Accuracy	Macro F1	Best Val Loss	Training Time	Epochs	Inference Time	Parameters
32	0.8127	0.8061	0.3161	11.65 s	38	0.0173 s	4678
64	0.8137	0.8074	0.3073	16.00 s	32	0.0594 s	17542
128	0.8174	0.8127	0.2850	48.08 s	27	0.3129 s	67846
256	0.8154	0.8093	0.3431	92.91 s	21	0.4518 s	266758

Table 3: LSTM with different hidden sizes (test metrics, seed 42).

Effect on performance. Changing the hidden size barely matters. Macro F1 stays between 0.806 and 0.813, which is less than the variation we saw between seeds (standard deviation 0.028). The best validation loss gets a little better up to 128 and then worse at 256, and that model also stops sooner (21 epochs). It looks like the largest LSTM starts to overfit or is harder to train, so going bigger does not reliably help.

Effect on cost. Cost, on the other hand, goes up a lot. Between 32 and 256 the parameters increase about 57 times (4,678 to 266,758), training time about 8 times (11.7 s to 92.9 s) and inference time about 26 times. One detail: the run with hidden size 64 does not match the LSTM from the main comparison (macro F1 0.807 vs. 0.787), even with the same seed. In the main comparison the four models are built one after another after a single `set_seed` call, so the LSTM gets a different initialisation. This is one more sign of how much the seed matters.

Conclusion. What holds the LSTM back is not its size but how the input is presented. Reading 561 engineered features one at a time as if they were a time series gives the recurrent model no real temporal structure to learn from. A small hidden size (32–64) gives the best balance between results and cost.